Docker and Kubernetes

Lesson Objectives

Session 1 – Getting started with Docker

Session 2 – Building Docker images

Session 3 – Docker Networking

Session 4 – Docker storage

Session 5 – Making a real project with Docker and NodeJs

Session 6 – Docker Swarm

Session 7-10 – Kubernetes

Session 11 – Final test

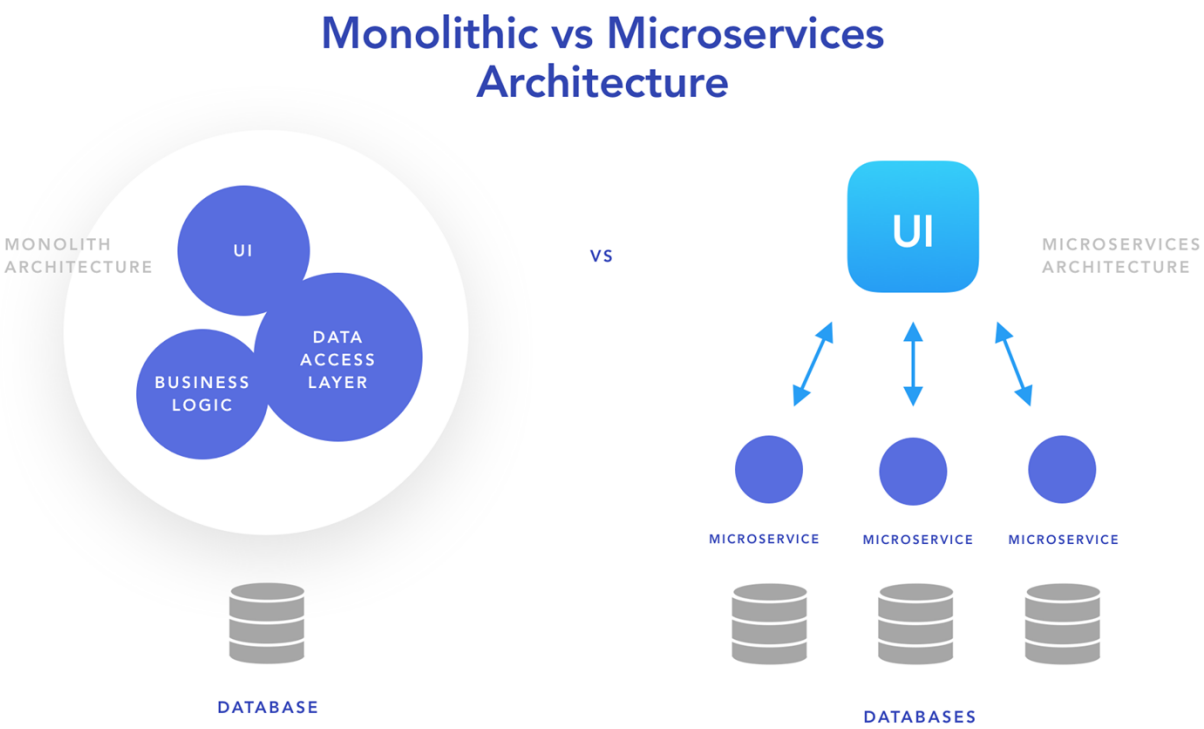
Session 7

Kubernetes

Agenda

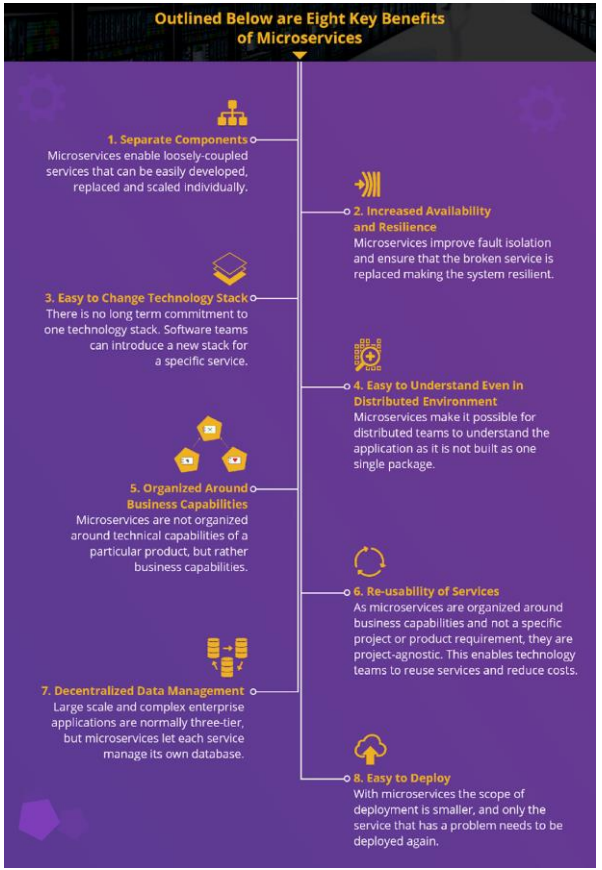
- Assignment Review & Guides
- From Monolithic to Microservices
- What is Kubernetes?
- Kubernetes architecture

From Monolithic to Microservices



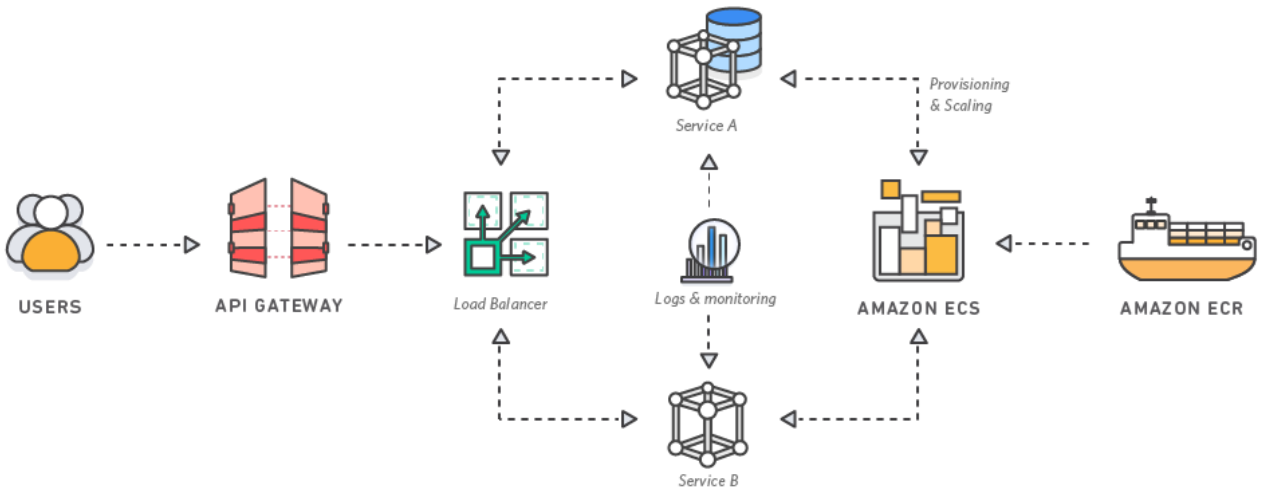
Benefits of microservices

Microservice architecture divides a single application into smaller sets of services, each running on its own but communicating with each other through APIs



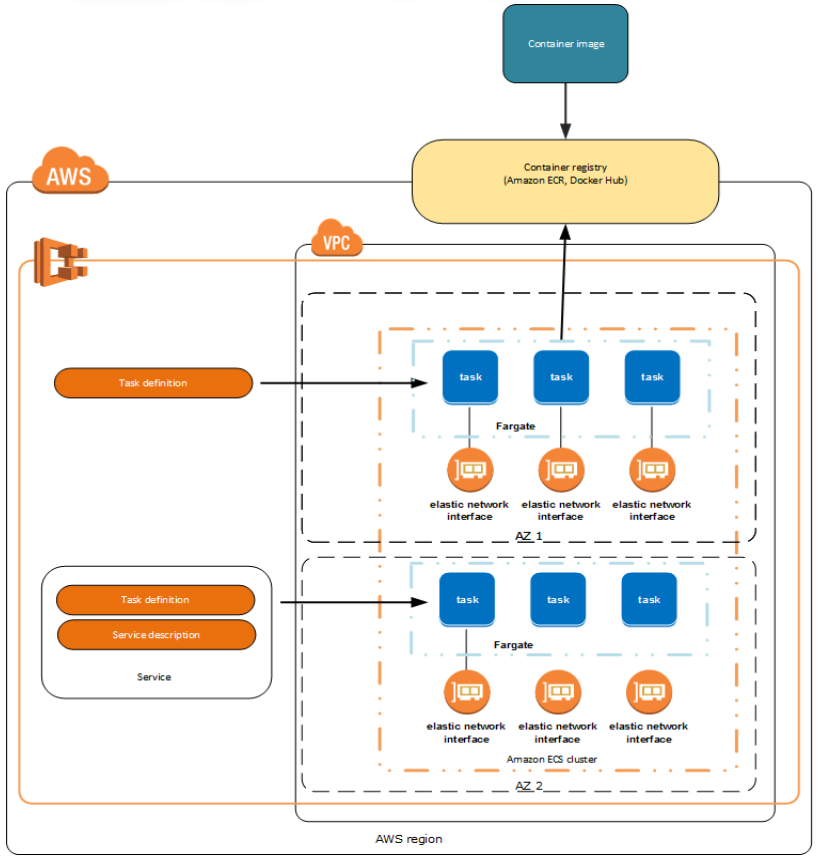
AWS ECS

Amazon Elastic Container Service (Amazon ECS) is a highly scalable, fast, container management service that makes it easy to run, stop, and manage Docker containers on a cluster of Amazon EC2 instances.



ECS Architecture

- Elements of the Amazon ECS:
- Cluster
- Container agent
- Containers and images
- Task definitions
- Task and scheduling



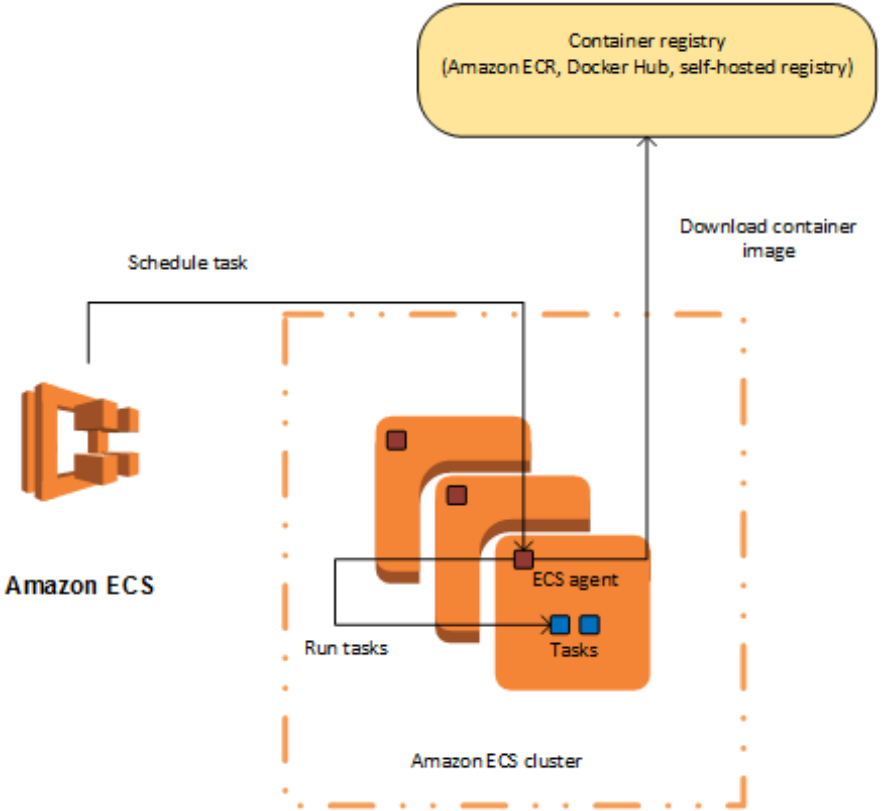
ECS – Cluster

- An Amazon ECS cluster is a logical grouping of tasks or services.

- You can register one or more Amazon EC2 instances, also referred to as container instances with your cluster to run tasks on, or you can use the serverless infrastructure that Fargate provides. When your tasks are run on Fargate, your cluster resources are managed by Fargate.

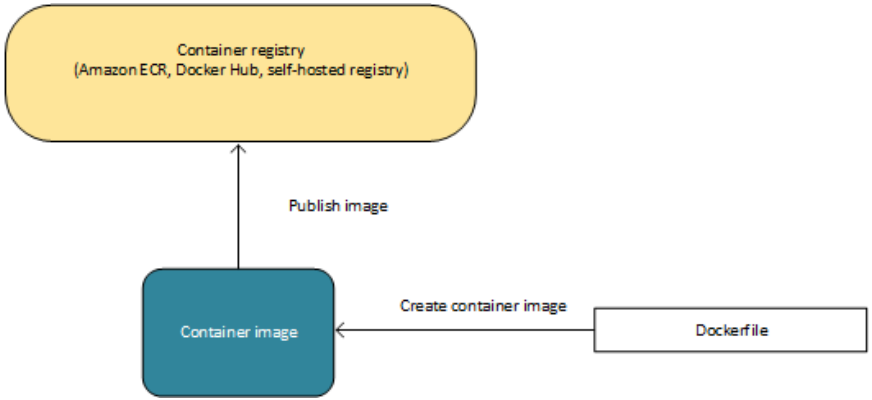
ECS – Container Agent

- The container agent runs on each container instance within an Amazon ECS cluster.
- It sends information about the resource's current running tasks and resource utilization to Amazon ECS, and starts and stops tasks whenever it receives a request from Amazon ECS



ECS – Container & Images

- To deploy applications on Amazon ECS, your application components must be architected to run in containers.
- A container is a standardized unit of software development, containing everything that your software application needs to run: code, runtime, system tools, system libraries, etc. Containers are created from a read-only template called an image.



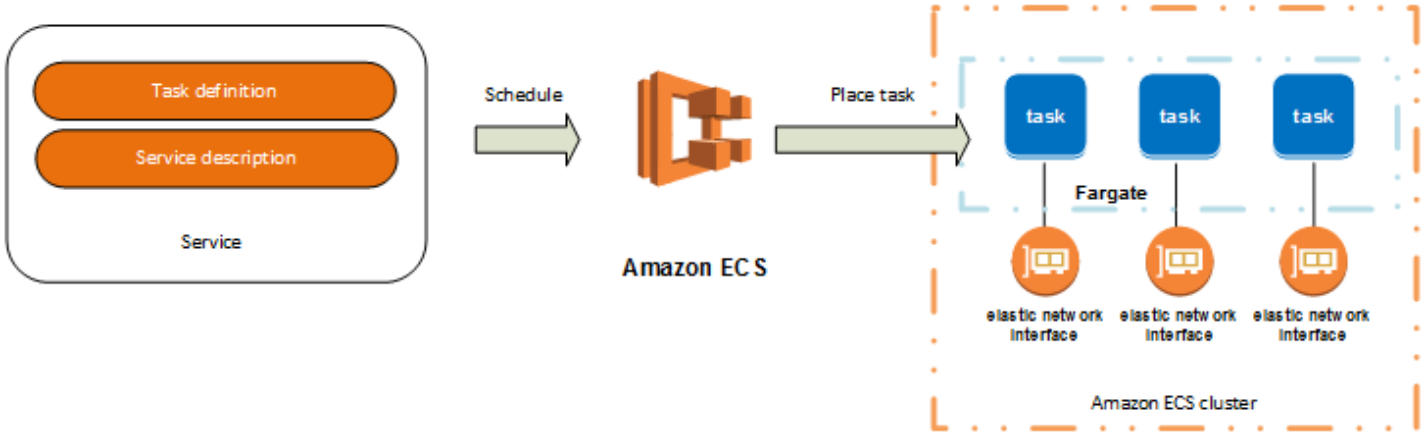
ECS – Task definitions

- To prepare your application to run on Amazon ECS, you create a task definition.
- The task definition is a text file, in JSON format, that describes one or more containers, up to a maximum of ten, that form your application

```
{
    "family": "webserver",
    "containerDefinitions": [
        {
            "name": "web",
            "image": "nginx",
            "memory": "100",
            "cpu": "99"
        },
    ],
    "requiresCompatibilities": [
        "FARGATE"
    ],
    "networkMode": "awsvpc",
    "memory": "512",
    "cpu": "256",
}
```

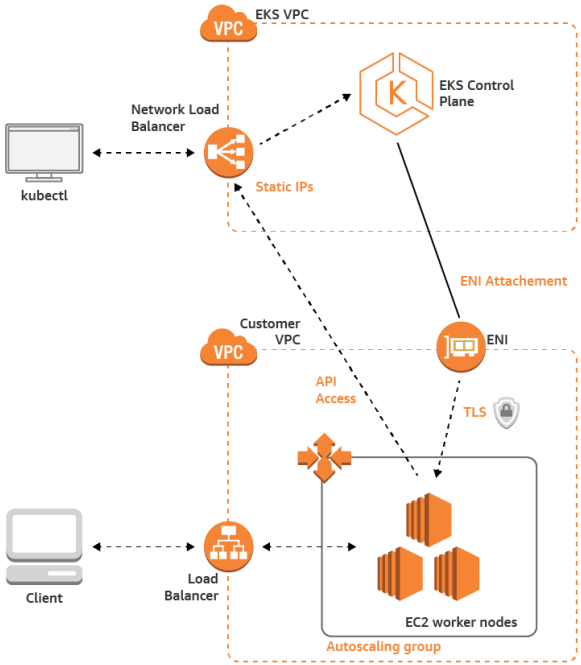
ECS – Tasks & Scheduling

- A task is the instantiation of a task definition within a cluster. After you have created a task definition for your application within Amazon ECS, you can specify the number of tasks that will run on your cluster.
- The Amazon ECS task scheduler is responsible for placing tasks within your cluster.



AWS EKS

- **Amazon EKS** is a managed service that makes it easy for you to run Kubernetes on AWS without needing to install, operate, and maintain your own Kubernetes control plane or nodes.
- **Kubernetes** is an open-source system for automating the deployment, scaling, and management of containerized applications

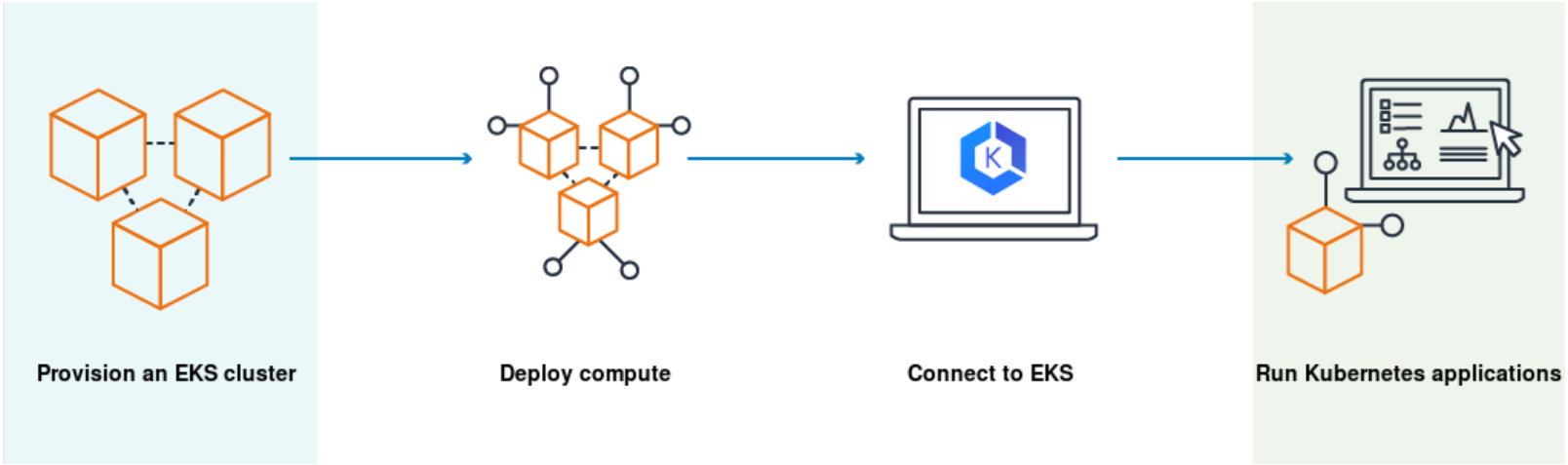


AWS EKS CONTROL PLANE

- Amazon EKS runs Kubernetes control plane instances across multiple Availability Zones to ensure high availability. Amazon EKS automatically detects and replaces unhealthy control plane instances, and it provides automated version upgrades and patching for them.

- Amazon EKS runs a single tenant Kubernetes control plane for each cluster, and control plane infrastructure is not shared across clusters or AWS accounts.

How does Amazon EKS work?



KUBERNETES

WHY CONTAINERS

- Build, Ship, and Run any App, Anywhere.
- No more dependency problems; e.x. no RPM/DEB or dynamic libraries incompatibility issues.
- No more: "but it works on my machine", as the same image used in dev is deployed to prod.
- Serverless implementations are mainly backed by containers.

WHY KUBERNETES

- Be able to manage hundreds or thousands of containers, on a fleet of hundred or thousands of nodes.
- Be able to deploy an application without worrying about the hardware infrastructure.
- Be able to guarantee that applications will stay running according with the specifications.
- Be able to facilitate scaling and upgrades not only for the workloads but for the k8s cluster itself.

WHAT IS KUBERNETES

- Kubernetes (K8s) is an open-source system for automating deployment, scaling, and management of containerized applications.
- Designed from the ground-up as a loosely coupled collection of components centered around deploying, maintaining and scaling workloads.
- Abstracts away the underlying hardware of the nodes and provides a uniform interface for workloads to be both deployed and consume the shared pool of resources.
- A distributed cluster technology that manages container-based systems in a declarative manner using an API (a container orchestrator).

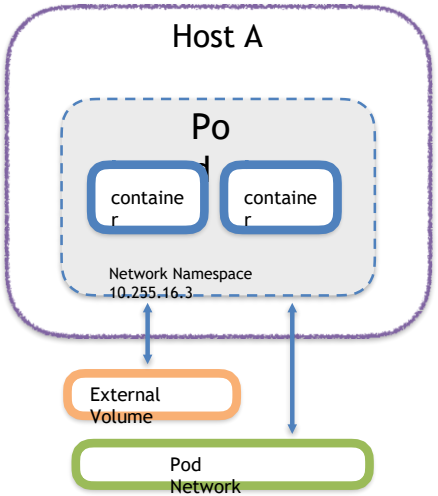
A COUPLE OF KEY CONCEPTS…

PODS

Ephemeral

Atomic unit or smallest “unit of work”of Kubernetes.

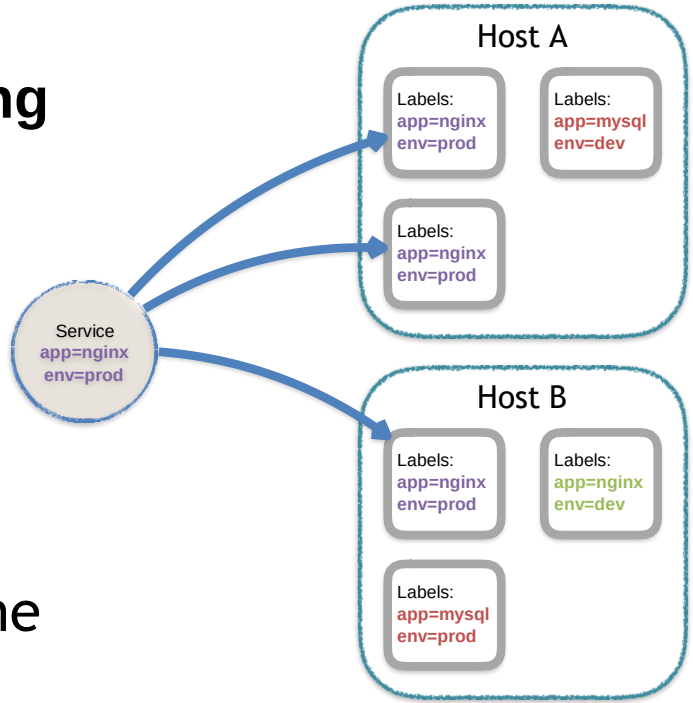
- Pods are **one or MORE containers** that share volumes, a network namespace, and are a part of a **single context**.



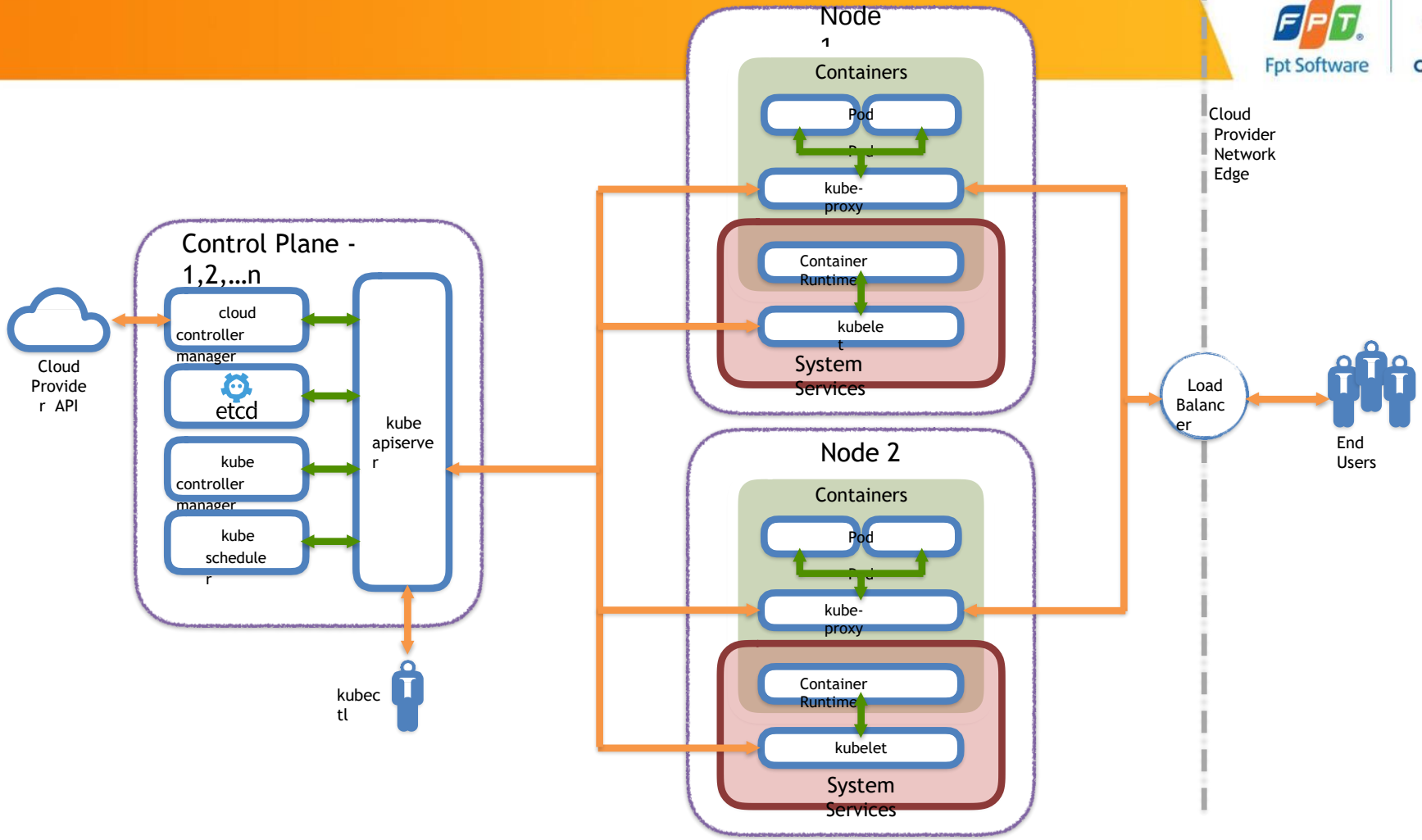
NOT Ephemeral

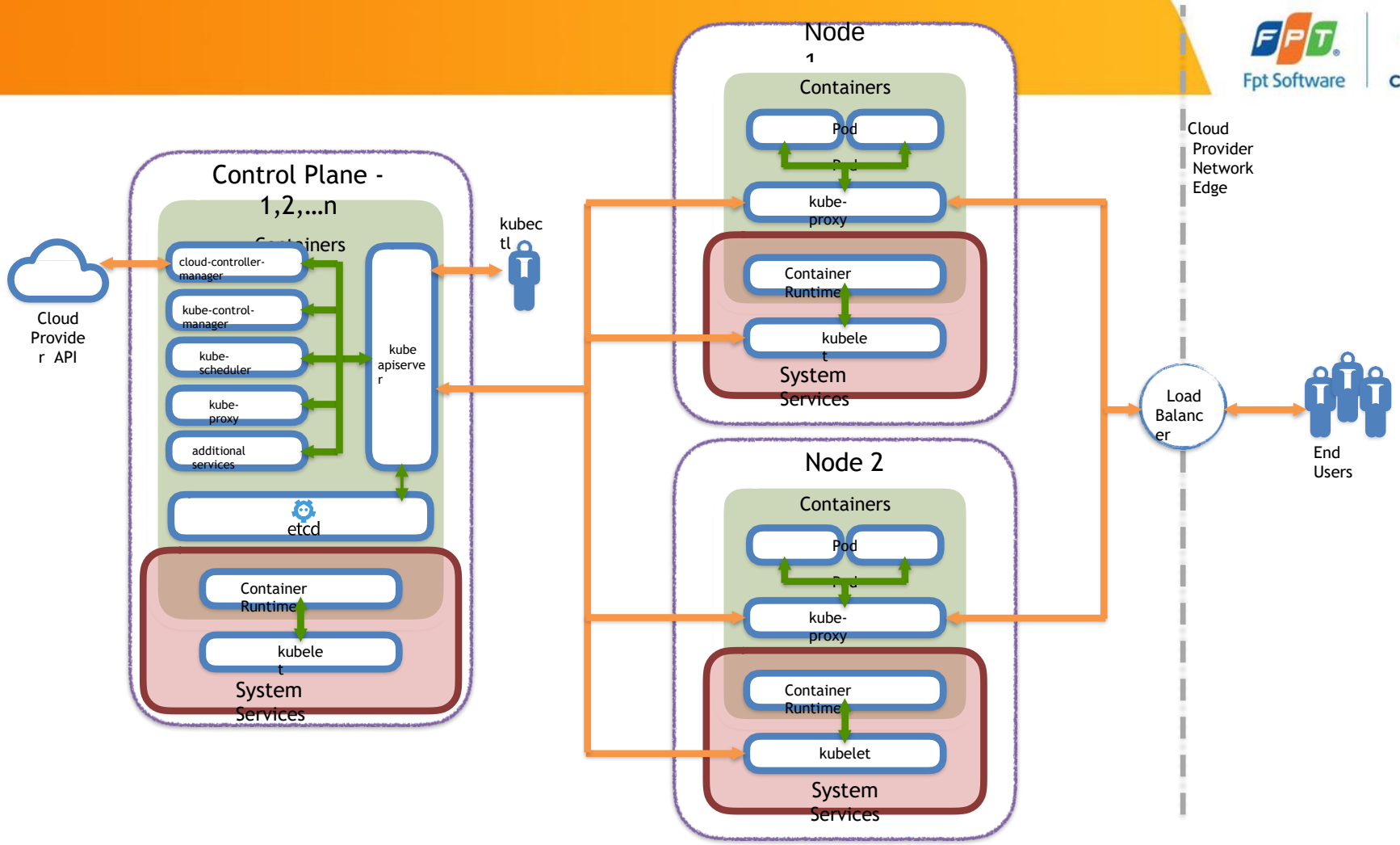
Unified method of accessing the exposed workloads of Pods.

- **Durable resource**
  - static cluster IP
  - static namespaced DNS name



ARCHITECTURE OVERVIEW


Node 1
Containers
Pod
kube-proxy
Container Runtime
kubelet
System Services
Cloud Provider Network Edge
Control Plane - 1,2,...n
cloud controller manager
etcd
kube controller manager
kube scheduler
kube apiserver
Cloud Provider API
kubectl
Node 2
Containers
Pod
kube-proxy
Container Runtime
kubelet
System Services
Load Balancer
End Users


Control Plane - 1,2,…n
Containers
cloud-controller-manager
kube-control-manager
kube-scheduler
kube-proxy
additional services
kube apiserver
etcd
Container Runtime
kubelet
System Services
kubectl
Cloud Provider API
Node 1
Containers
Pod
kube-proxy
Container Runtime
kubelet
System Services
Node 2
Containers
Pod
kube-proxy
Container Runtime
kubelet
System Services
Cloud Provider Network Edge
Load Balancer
End Users

KUBE-APISERVER

- Provides a forward facing REST interface into the kubernetes control plane and datastore.
- All clients and other applications interact with kubernetes **strictly** through the API Server.
- Acts as the gatekeeper to the cluster by handling authentication and authorization, request validation, mutation, and admission control in addition to being the front-end to the backing datastore.

ETCD

- etcd acts as the cluster datastore.
- Purpose in relation to Kubernetes is to provide a strong, consistent and highly available key-value store for persisting cluster state.
- Stores objects and config information.
- Uses “Raft Consensus” among a quorum of systems to create a fault-tolerant consistent “view” of the cluster.

KUBE-CONTROLLER-MANAGER

- Serves as the primary daemon that manages all core component control loops.

- Monitors the cluster state via the apiserver and steers the cluster towards the desired state.

KUBE-SCHEDULER

- Component on the master that watches newly created pods that have no node assigned, and selects a node for them to run on.

- Factors taken into account for scheduling decisions include individual and collective resource requirements, hardware/software/policy constraints, affinity and anti-affinity specifications, data locality, inter-workload interference and deadlines.

CLOUD-CONTROLLER-MANAGER

master

- Daemon that provides cloud-provider specific knowledge and integration capability into the core control loop of Kubernetes.

- The controllers include Node, Route, Service, and add an additional controller to handle things such as **PersistentVolume** Labels.

KUBE-PROXY

worke
r

- Manages the network rules on each node.
- Performs connection forwarding or load balancing for Kubernetes cluster services.

KUBELET

- An agent that runs on each node in the cluster. It makes sure that containers are running in a pod.

- The kubelet takes a set of PodSpecs that are provided through various mechanisms and ensures that the containers described in those PodSpecs are running and healthy.

CONTAINER RUNTIME ENGINE

A container runtime is a CRI (Container Runtime Interface) compatible application that executes and manages containers.

- **containerd** (Docker)
- **cri-o**
- **rkt**
- **kata** (formerly clear and hyper)
- **virtlet** (VM CRI compatible runtime)

- **Pod Network**
  - Cluster-wide network used for pod-to-pod communication managed by a CNI (Container Network Interface) plugin.

- **Service Network**
  - Cluster-wide range of Virtual IPs managed by **kube-proxy** for service discovery.

FUNDAMENTAL NETWORKING RULES

- All containers within a pod can communicate with each other unimpeded.
- All Pods can communicate with all other Pods without NAT.
- All nodes can communicate with all Pods (and vice-versa) without NAT.
- The IP that a Pod sees itself as is the same IP that others see it as.

What if I want to limit communication within pods?

Learn about **Network Policies** (and make sure the chosen CNI supports it)

Q&A